

## MODULE 2

# DESIGNING LLM APPLICATIONS

1

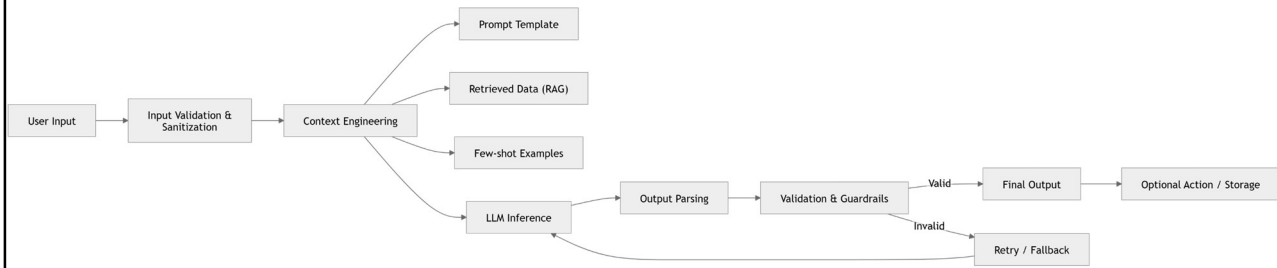
### Purpose

- > Build production-ready LLM applications
- > Transform user problems into model-executable tasks
- > Design reliable, controllable, and scalable pipelines
- > Move from prompt engineering → system engineering

2

## Introduction

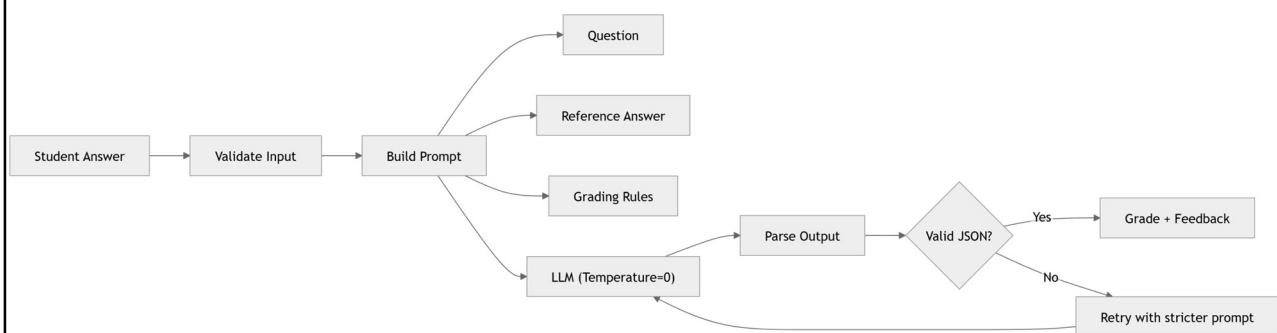
- > Designing LLM applications is not about writing prompts.
- > It is about **engineering a reliable, controllable, and scalable system around a probabilistic model.**
- > At a high level, you are designing a pipeline that transforms:



3

## Example

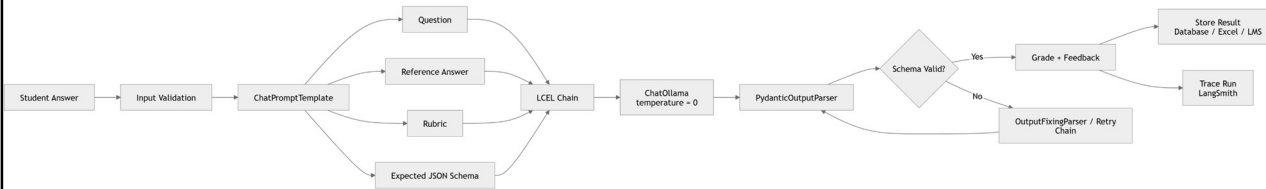
- > Quiz Grading LLM Application



4

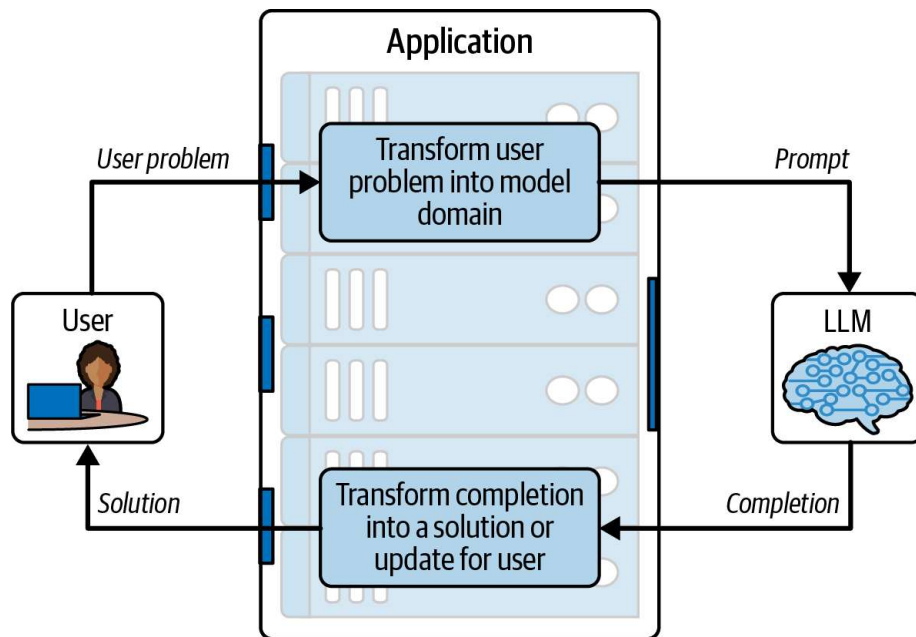
## Example

### > LangChain Quiz Grading architecture



5

## The Anatomy of the Loop



6

## The User's Problem

|                              | Increasing complexity →                                                      |                                                                                                       |                                                                                                                                      |
|------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
|                              | Proofreading                                                                 | IT support assistance                                                                                 | Travel planning                                                                                                                      |
| <i>Medium of the problem</i> | Text                                                                         | Voice over the phone                                                                                  | Complex interactions on the website, text input from the user, and interactions with APIs.                                           |
| <i>Level of abstraction</i>  | The problem is concrete, well-defined, and small.                            | A large abstract problem space and a large solution space, but constrained by available documentation | The problem involves understanding the user's subjective tastes and objective constraints in order to coordinate a complex solution. |
| <i>Context required</i>      | Nothing more than the text submitted by the user.                            | Searchable access to technical documentation and example support transcripts                          | Access to calendars, airlines APIs, recent news articles, government travel recommendations, Wikipedia, etc.                         |
| <i>Statefulness</i>          | No statefulness—every call to the API contains a distinct problem statement. | Must track the conversation history and solutions attempted                                           | Must track interaction across weeks of planning, different mediums of interaction, and aborted branches of planning.                 |

7

## Converting the User's Problem to the Model Domain

- > The goal is to create a prompt so that its completion contains information that can be used to address the user's problem.
- > Crafting just the right prompt is quite a tall order, and the application must satisfy the following criteria simultaneously:
  1. The prompt must closely resemble content from the training set.
  2. The prompt must include all the information relevant to addressing the user's problem.
  3. The prompt must lead the model to generate a completion that addresses the problem.
  4. The completion must have a reasonable end point so that generation comes to a natural stop.

8

## Key design dimensions: Problem framing

- > Before coding anything
  - Is this a **generation task** (write, summarize)?
  - Or a **decision task** (classify, grade)?
  - Or a **reasoning task** (multi-step logic)?
- > Different tasks require different control strategies.
- > Example:
  - grading → deterministic, structured
  - chat → flexible, conversational

9

## Key design dimensions: Prompt design (not just text)

- > Prompts should be treated as:
  - interfaces**, not strings
- > A good prompt defines:
  - role (who the model is)
  - task (what to do)
  - constraints (how to behave)
  - output format (how to respond)
- > Example structure:
  - Role: strict grader
  - Task: evaluate answer
  - Constraints: be consistent, no hallucination
  - Output: JSON

10

## Key design dimensions: Context engineering (most critical)

- > LLMs don't "know"—they **condition on context**.
- > You must decide:
  - what information to include
  - what to exclude
  - how to structure it
- > Techniques:
  - RAG (retrieval augmented generation)
  - few-shot examples
  - instruction hierarchy

11

## Key design dimensions: Determinism vs creativity

- > You control behavior via:
  - temperature
  - top\_p
  - max tokens
- > For applications like grading:
  - low temperature ( $\approx 0.0$ )
- > For creative writing:
  - higher temperature

12

## Key design dimensions: Output control

> Never trust raw LLM output in production.

> You should enforce:

- schemas (JSON output)
- validation rules
- Parsing

> Example:

```
{  
  "grade": 8,  
  "feedback": "..."  
}
```

13

## Key design dimensions: Error handling & robustness

> LLMs can

- hallucinate
- ignore instructions
- produce malformed outputs

> So, you need

- retries
- fallback prompts
- output validation
- guardrails

14

## Key design dimensions: Tool usage (augmentation)

- > LLMs alone are limited.
- > You extend them with:
  - APIs
  - databases
  - calculators
  - vector search
- > This turns the system into  
**LLM + tools = intelligent system**

15

## Key design dimensions: Memory and state

- > For conversational systems:
  - short-term memory → chat history
  - long-term memory → vector DB
- > Without memory:
  - stateless system
- > With memory:
  - adaptive system

16



## Key design dimensions: Evaluation (often ignored)

- > You must measure
  - accuracy
  - consistency
  - hallucination rate
  - latency
- > For grading systems
  - inter-run consistency is critical

17

## Key design dimensions: Security (very important)

- > LLMs are vulnerable to:
  - prompt injection
  - data leakage
  - adversarial inputs
- > Design must include:
  - input filtering
  - context isolation
  - output sanitization

18

## Design patterns

- > **Single-step chain**
  - Simple prompt → model → output
- > **RAG pipeline**
  - retrieve → augment → generate
- > **Tool-augmented system**
  - LLM decides → calls tool → integrates result
- > **Agent system**
  - LLM iteratively plans and acts

19

## Common mistakes

- > Treating LLM like a deterministic function
- > Overloading prompts with too much context
- > Not validating outputs
- > Ignoring latency and cost
- > Not designing for failure

20

## Langchain

- > LangChain is not a model
- > It is an **orchestration framework** for building applications around LLMs.
- > Think of it as an abstraction layer that helps you move from **single prompt** → **full AI system**.

21

## Langchain: Core idea

- > At a conceptual level, LangChain solves this problem:
  - How do we structure, compose, and control LLM interactions in a reliable way?*
- > Instead of writing ad-hoc prompt code, LangChain introduces **modular components**.

22

## Langchain: Models

- > These are the LLM interfaces.
  - OpenAI, Ollama, HuggingFace, etc.
  - Chat vs completion models
- > LangChain standardizes different providers under one API

23

## Langchain: Prompts

- > Prompts are treated as **first-class objects**, not strings.
- > They allow:
  - templating
  - variable injection
  - structure enforcement
- > Example:

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Explain {topic} in simple terms"
)
```
- > This enables reusable and parameterized prompting

24

## Langchain: Chains

- > Chains are **pipelines of operations**.
- > You define
  - prompt → model → output parser
- > Example:

```
chain = prompt | llm
result = chain.invoke({"topic": "self-attention"})
```
- > This is functional composition

25

## Langchain: Output Parsers

- > LLMs produce text, but applications need **structured data**.
- > Parsers convert:
  - text → JSON
  - text → objects
- > Example:

```
from langchain_core.output_parsers import StrOutputParser
chain = prompt | llm | StrOutputParser()
```

26

## Langchain: Memory

- > Memory enables **stateful conversations**.
- > Types:
  - short-term (chat history)
  - long-term (vector stores)
- > Without memory: stateless API
- > With memory: conversational agent

27

## Langchain: Retrievers (RAG)

- > Retrievers enable **external knowledge access**
- > Pipeline:
  - query → retrieve docs → augment prompt → LLM
- > This is Retrieval-Augmented Generation (RAG)

28

## Langchain: Tools

- > Tools allow LLMs to **interact with external systems**:
  - APIs
  - databases
  - calculators
- > Example:
  - LLM decides: “I need to call a tool”

29

## Langchain: Agents

- > Agents are the most advanced abstraction.
- > They:
  - reason
  - decide actions
  - call tools
  - iterate
- > Chain = fixed pipeline
- > Agent = dynamic decision-making system

30

## Langchain: Execution model

> LangChain uses a **dataflow model**:

–Input → Prompt → Model → Parser → Output

> or more complex:

–User → Retriever → Prompt → LLM → Tool → LLM → Output

> This is why LCEL (LangChain Expression Language) exists:

chain = prompt | llm | parser

31

## Langchain: Minimal mental model

> Think of LangChain as:

–**TensorFlow for LLM pipelines**

- Models = layers
- Chains = graphs
- Prompts = inputs
- Tools = external operators

32



## Simple architecture example

```
prompt = PromptTemplate.from_template(  
    "Summarize: {text}"  
)  
  
chain = prompt | llm  
  
chain.invoke({"text": "..."})
```

33

## Why LangChain matters

> From an engineering perspective:

### –Without LangChain

- spaghetti prompt code
- no composability
- hard to scale

### –With LangChain

- modular architecture
- reusable components
- testable pipelines
- easier integration

34